LHCB UPGRADE II · PICOCAL R&D TECHNICAL USER MANUAL & REFERENCE GUIDE

Conical Light Guide Inner-Surface Imperfection Simulation, Inverse Reconstruction & 3D Visualization Suite (Agy_Carbon)

Heterogeneous Metal GPU / Multi-Core C++17 Specular B-Spline Ray Tracing, Rotationally Equivariant Gauss-Newton Surface Inversion, and Geant4 11 Optical Validation

Author: Hengne Li (李衡讷) — South China Normal University (SCNU) / LHCb Collaboration, CERN
Contact: Hengne.Li@cern.ch, Hengne.Li@m.scnu.edu.cn | **Document Edition:** v2.0 Cross-Platform Bilingual Release

0.314 mV (1.24%)
Inner-Aperture Net RMSE (vs 0.81% Poisson Noise Floor)

r = 0.9931
Full-Map 2D Pearson Correlation (Bump r = 0.8797)

~300M rays/s
Apple Metal GPU Spline Throughput (~7.5 s Full Inversion)

< 380 MB
Peak Physical RAM via MemoryGuard (Solver RSS 335 MB)

Table of Contents & Manual Navigation

Chapter 1 — Scientific Background & Physical Mechanism: 4-cell STEP CAD geometry, 2D laser scan calibration, optical Etendue limit (20.9%), and critical-angle turnaround amplification.

Chapter 2 — Surface Parameterization & Inverse Solver Architecture: Periodic-in-θ Bicubic B-Spline R(θ, z), C ≡ 0 concentric reference cone root bracket, and 3-stage SO(2) Gauss-Newton solver.

Chapter 3 — System Requirements & Cross-Platform Setup: Path A lightweight Python + C++17/Metal setup (Linux/macOS/Windows) and Path B full Conda + Geant4 11 particle-physics suite.

Chapter 4 — Step-by-Step Guide to Reproducing the Full Simulation: One-click run-all.sh workflow, modular execution, stage-by-stage terminal log walkthrough, and CPU/GPU runtime benchmarks.

Chapter 5 — Reconstructing Your Own Light-Guide Scan Dataset: Required CSV schema, coordinate conventions, automatic sub-pixel calibration, CLI flags, and solver tuning guide.

Chapter 6 — Interactive Web Portals, Zoomable SVGs & 3D Photon Bench: Using bilingual portals, single-file standalone HTML bundles, 24× zoomable vector dashboards, and 3D photon simulator controls.

Chapter 7 — Repository Dictionary, Output Schemas & Troubleshooting FAQ: Complete directory map, JSON/CSV/STL output data dictionary, MemoryGuard behavior, compiler tips, and Geant4 dataset variables.

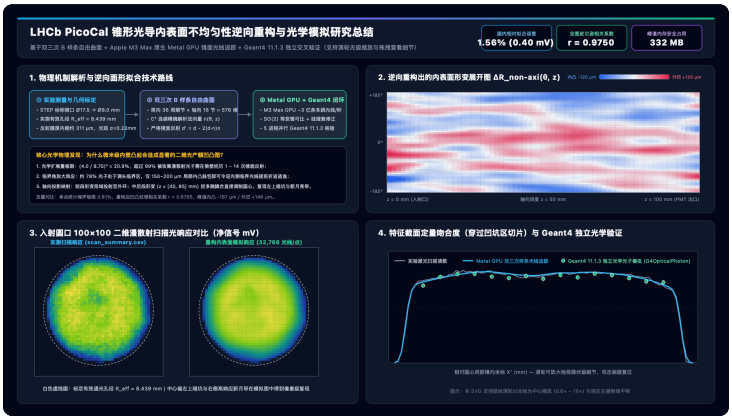


Figure 0.1 — Executive Research Summary Board: Closed loop from STEP CAD parsing and 100 × 100 diffuse laser uniformity calibration to 576-DOF periodic bicubic B-spline surface inversion and 5-way parallel Geant4 11 optical-photon verification.

Chapter 1. Scientific Background & Physical Mechanism

1.1 LHCb PicoCal 4-Cell Conical Light Guide Geometry & Bench Scan Calibration

High-granularity electromagnetic calorimetry in the **LHCb Upgrade II PicoCal** relies on hollow specularly reflective conical light guides to funnel scintillation or Cherenkov photons from front converter cells onto fast downstream photomultiplier tubes (PMTs) while preserving picosecond timing and spatial uniformity. The prototype studied in this suite (`data/4cells_17.5mm_8mm_circle_to_circle_100mm-Cut.step`) is a 4-channel CNC-machined block measuring `49.0 × 49.0 × 110.0 mm` with four parallel cell axes centered at `(±10.0, ±10.0) mm`.

Subsystem	Physical Quantity	Nominal / Calibrated Value	Physical Interpretation & Method
STEP CAD	Axial Reflective Length `L`	`100.00 mm` (`z ∈ [0, 100] mm`)	Extracted automatically from `CONICAL_SURFACE` & `CIRCLE` entities
STEP CAD	Entrance / Exit Radii	`R_in = 8.75 mm` → `R_out = 4.00 mm`	`Ø17.5 mm` entrance to `Ø8.0 mm` exit; slope `−0.0475` , half-angle `α = 2.72°`
STEP CAD	PMT Mounting Bore	`R_bore = 8.18 mm` (`Ø16.36 mm`)	Downstream counterbore spanning `z ∈ [100.0, 110.0] mm`
2D Bench Scan	XY Raster Grid	`100 × 100 = 10,000` pixels (`0.20 mm` step)	Stage window `X ∈ [−7.5, +12.3] mm` , `Y ∈ [−14.5, +5.3] mm` ; 30 triggers/pt
2D Bench Scan	Calibrated Center & `R_eff`	`(2.3665, −4.3788) mm` , `R_eff = 8.438 mm`	`~312 µm` inward offset from `8.750 mm` due to inner reflective film thickness
2D Bench Scan	Spot Size & Noise Floor	`σ_beam = 0.219 mm` , noise `0.81%`	Pedestal `15.243 mV` ; unbiased 2D discrete Laplacian noise floor inside `r < 7.5 mm`

1.2 Phase-Space Etendue Limit & Critical-Angle Turnaround Amplification

In the experimental uniformity testbench, a localized diffuse laser probe scans across the `z = 0` entrance plane, emitting photons into the forward hemisphere with an approximately Lambertian angular distribution (`I(θ) ∝ cos θ`). Unlike paraxial point-to-point imaging, a concentrator with compression ratio `R_in / R_out = 8.75 / 4.00 = 2.1875` is governed strictly by 3D optical phase-space conservation (**Liouville's theorem / Etendue**):

$$\eta_{\text{ideal, max}} \leq (R_{\text{out}} / R_{\text{in}})^2 = (4.00 / 8.75)^2 \approx 20.90\%, \quad \sin\theta_{\text{crit}} \approx R_{\text{out}} / R_{\text{in}} = 0.4571 \Rightarrow \theta_{\text{crit}} \approx 27.2^\circ \tag{1.1}$$

Every time a ray reflects off the tapered cone wall (half-angle `α = 2.72°`), its polar angle relative to the `+z` optical axis increases by approximately `2α = 5.44°` (modulated by skew angular momentum for off-axis rays). Detailed Monte Carlo tracking reveals three decisive physical regimes:

- **Multi-Bounce Dominance (>99% of Collected Photons):** Direct unreflected solid angle subtends less than `0.2%` ; virtually all collected photons experience **1 to 14 specular wall reflections** (with effective film reflectivity `R_film ≈ 95.5%`).
- **Massive Turnaround Reservoir (~78% of Launched Rays):** Rays emitted with polar angles above `θ_crit ≈ 27.2°` accumulate polar angle across successive bounces until `θ > 90°` (`d_z < 0`), turning around mid-cone and escaping backward out of the `z = 0` entrance port.
- **Critical-Angle Turnaround Amplification:** Because a dense population of marginal rays propagates right on the knife-edge of turnaround (`θ ∈ [24°, 28°]`), a microscopic inward surface bulge of `ΔR ≈ −268 µm` (hand-laminated ESR/Mylar film blister or machining waviness) locally steepens the upstream windward slope `|∂R/∂z|` . Marginal rays striking this windward ramp gain an extra polar deflection `2Δα` , flipping them from successful PMT transmission into premature backward escape. Conversely, striking the leeward gentler slope preserves marginal rays, generating paired bright crescents (`+4.5%`) adjacent to dark dips (`−5.5%`).

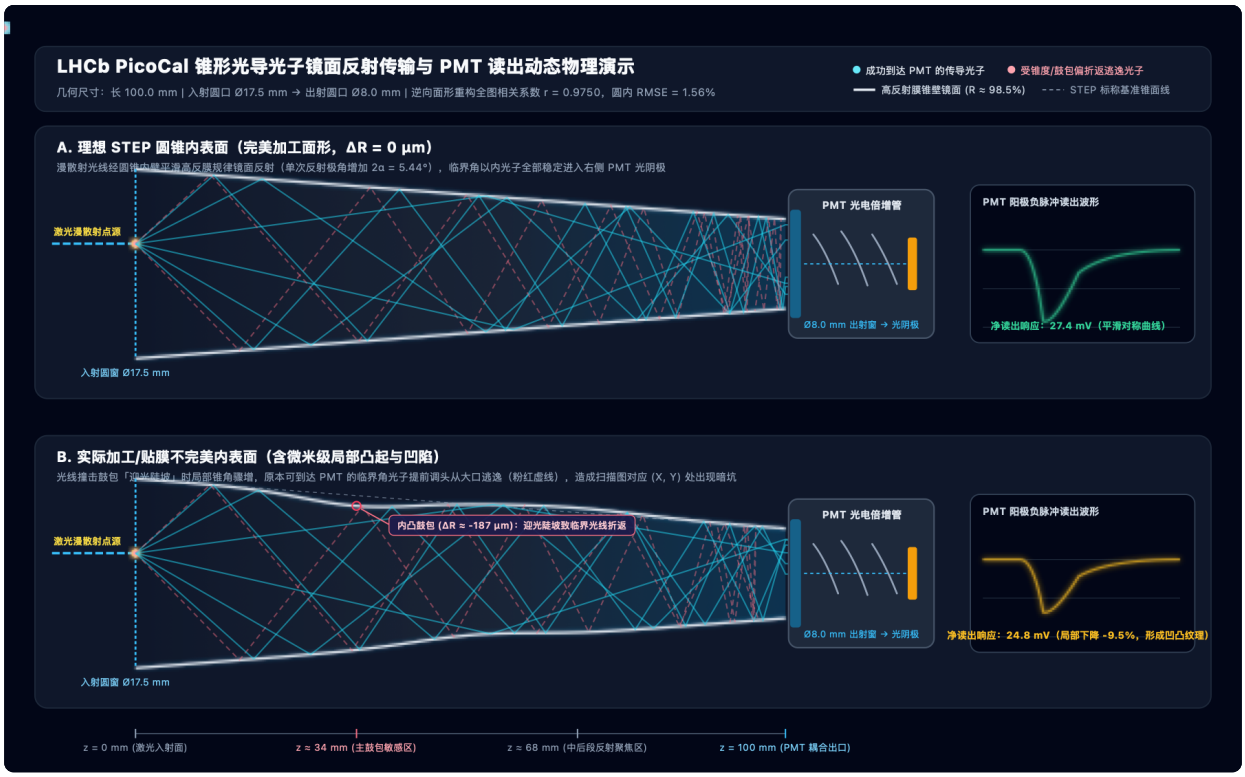


Figure 1.1 — Specular Photon Transport & Critical-Angle Turnaround Escape: (Top) Ideal STEP cone (`ΔR = 0 μm`) showing orderly polar-angle progression into the Ø8.0 mm PMT window (`27.4 mV`). (Bottom) Reconstructed deformed wall containing a `−268 μm` inward bulge near `z ≈ 34 mm` ; rays hitting the steeper upstream ramp reverse direction and escape backward (coral dashed trajectories), producing the localized readout dip (`24.8 mV`).

Chapter 2. Mathematical Surface Parameterization & Inverse Solver Architecture

2.1 Periodic-in-θ Bicubic B-Spline Freeform Surface Model

Representing curved optical mirrors with raw triangle facets introduces discontinuous surface normals across facet edges, corrupting specular caustics with artificial polygonal banding. Instead, `surface-model.py` parameterizes the inner reflective wall in cylindrical coordinates `(θ, z) ∈ [−π, π) × [0, L]` as an orthogonal decomposition over a uniform **Periodic-in-θ Bicubic B-Spline tensor product**:

$$R(\theta, z) = R_{\text{CAD}}(z) + \Delta R_{\text{axi}}(z) + \Delta R_{\text{non-axi}}(\theta, z) = R_{\text{in}} + s_0\, z + \sum_{a=0}^{3} \sum_{b=0}^{3} C_{(i_0+a-1) \bmod N_\theta,\, j_b}\, B_a(f_u)\, B_b(f_v) \tag{2.1}$$

where `s_0 = (R_out − R_in) / L = −0.0475` is the nominal CAD cone slope, `N_θ = 36` circumferential control knots (`10°` spacing) wrap modulo `36` to guarantee strict `C^2` **curvature continuity** across the `θ = ±π` branch cut, and `N_z = 16` clamped axial knots span `z ∈ [0, 100] mm` . Enforcing zero circumferential mean on every axial slice (`∑_i ΔR_non-axi(i, j) = 0`) orthogonally decouples the **16 axisymmetric profile DOFs** (overall film thickness & axial bow) from the `36 × 16 = 576` **non-axisymmetric bump/dent DOFs**.

At any ray-wall intersection `(x, y, z)` with `r = √(x^2 + y^2)` and `θ = atan2(y, x)` , differentiating the implicit level-set `F(x, y, z) = r − R(θ, z) = 0` yields exact analytical partial derivatives `(∂R/∂θ, ∂R/∂z)` and the exact inward unit surface normal `n` :

$$\nabla F = (\cos\theta + (\sin\theta / r)(\partial R/\partial\theta),\ \sin\theta - (\cos\theta / r)(\partial R/\partial\theta),\ -\partial R/\partial z),\quad \mathbf{n} = -\nabla F / \|\nabla F\|,\quad \mathbf{d}' = \mathbf{d} - 2(\mathbf{d}\cdot\mathbf{n})\mathbf{n} \tag{2.2}$$

2.2 Self-Intersection-Free Concentric Reference Cone Root Solver (C ≡ 0)

A subtle numerical trap arises during multi-bounce ray tracing inside deformed cones: if a ray reflects at an outward-dented wall point where `r_0 = R(θ_0, z_0) > R_CAD(z_0)` , initializing the next bounce search from the nominal CAD cone produces a negative quadratic constant `C = r_0^2 − R_CAD(z_0)^2 > 0` whose smallest positive root snaps backward onto the current bounce point (spurious `t ≈ 0` self-intersection). Both the Metal GPU shader (`ray-tracer-metal.mm`)

and C++17 kernel (`ray-tracer-native.cc`) eliminate this analytically by constructing a **dynamic concentric reference cone passing directly through the current bounce radius** `(z_0, r_0)` with nominal slope `s_0` :

$$R_{\text{ref}}(t) = r_0 + s_0\, d_z\, t \;\Rightarrow\; A\,t^2 + B\,t + C = 0 \;\text{ with }\; C \equiv x_0^{\,2} + y_0^{\,2} - r_0^{\,2} = 0 \;\Rightarrow\; t_{\text{init}} = -B / A \tag{2.3}$$

Because the `t = 0` root factors out identically, `t_init = -B / A` jumps cleanly across the chord of the guide to the true opposite/downstream wall impact! Four damped Newton-Raphson iterations along the ray `g(t) = r(t) - R(θ(t), z(t)) = 0` then converge to **sub-micrometer accuracy (`|g(t)| < 0.5 μm`)**.

2.3 Three-Stage SO(2) Equivariant Gauss-Newton Inverse Solver & Geant4 Cross-Check

Directly computing finite-difference sensitivities across `576` non-axisymmetric B-spline control nodes on a 2D scan grid would require `2 × 576 = 1,152` full Monte Carlo ray-tracing passes per iteration. `surface-inverter.py` exploits the exact `SO(2)` **rotational equivariance** of the axisymmetric base state to accelerate Jacobian assembly by `36×` :

1. **Stage 1 — 1D Axisymmetric Baseline Calibration (~0.9 s):** Fits the 16 axial nodes `ΔR_axi(z)` and overall photodetector scale `G_mV` against the azimuthally averaged radial curve `V_meas(r)` using `16,384` scrambled Sobol QMC rays/point (reconstructing `ΔR_axi ∈ [-462.1, -137.8] μm`).
2. **Stage 2 — SO(2) Equivariant Sensitivity Matrix Synthesis (~2.9 s):** Evaluates central differences (`±65 μm`) ONLY along the single meridian `i = 0` (`16 × 2 = 32` forward passes on a centered `42 × 42` grid) and synthesizes all remaining `35 × 16 = 560` Jacobian columns via bicubic 2D rotational affine transforms `R(2πi / 36)` .
3. **Stage 3 — Curvature-Regularized Gauss-Newton with Nonlinear Backtracking (~2.5 s):** Solves the damped normal equations with anisotropic biharmonic Laplacian penalties in `(θ, z)` :

   $$(J^{\mathrm{T}}J + \lambda_\theta L_\theta^{\mathrm{T}} L_\theta + \lambda_z L_z^{\mathrm{T}} L_z + \lambda_0 I)\,\Delta\mathbf{c} = J^{\mathrm{T}}\,(\mathbf{y}_{\text{meas}} - \mathbf{y}_{\text{sim}}) - \gamma\, R_{\text{reg}}\,\mathbf{c} \tag{2.4}$$

   followed at every step by an exact nonlinear ray-tracing backtracking line search over `α ∈ {0.65, 0.38, 0.20, 0.10}` to guarantee monotonic true-residual descent across caustic nonlinearities.
4. **Stage 4 — Independent High-Statistics Validation & Geant4 11 Cross-Check:** Evaluates the full `100 × 100` grid with an independent scrambled Sobol seed (`32,768` rays/pixel = `3.28 × 10^8` rays), then exports a closed watertight `G4TessellatedSolid` mesh to `geant4-validator/geant4-light-guide.cc` and tracks `15 × 8,000 = 120,000` independent `G4OpticalPhoton` trajectories across `Y′ = +1.6 mm` (5 parallel processes, yielding mean `25.21 mV` across `[23.83, 26.84] mV`).

Chapter 3. System Requirements & Step-by-Step Environment Setup

`Agy_Carbon` is engineered for seamless portability across particle-physics Linux clusters, macOS Apple Silicon / Intel laptops, and Windows x64 workstations. Its runtime dispatcher (`ray-tracer.py`) automatically selects the fastest compiled hardware backend available and falls back gracefully without code changes.

Operating System / Architecture	Primary Ray-Tracing Backend	Compiler Used	Geant4 11 Support
macOS Apple Silicon (M1/M2/M3/M4)	Native Metal GPU Shader (`libmetal_raytracer.dylib`)	`clang++ -ObjC++ -framework Metal`	Full (via `conda-forge`)
Linux (x86_64 / aarch64)	Multi-Core C++17 Thread Pool (`libraytracer.so`)	`g++` or `clang++ -O3 -pthread`	Full (via `conda-forge` / CVMFS)
macOS Intel (x86_64)	Multi-Core C++17 Thread Pool (`libraytracer.dylib`)	`clang++ -O3 -std=c++17`	Full (via `conda-forge`)
Windows 10/11 (x64)	C++17 DLL (`libraytracer.dll`) or PyTorch Vectorized	MinGW `g++` / MSVC `cl` / Optional	Optional (use `--skip-geant4`)

3.1 Path A (Fastest Start): Lightweight Python + C++17 / Metal Accelerator (No Geant4 Required)

Choose **Path A** if you want to run the complete inverse surface reconstruction, export 3D CAD STL models, and generate all interactive HTML/SVG/PDF deliverables within 2 minutes of setup, without downloading multi-gigabyte nuclear/optical cross-section datasets:

```
# 1. Enter the suite directory and create an isolated Python >= 3.10 virtual environment
cd Agy_Carbon
python3 -m venv .venv
source .venv/bin/activate          # On Windows PowerShell: .venv\Scripts\Activate.ps1

# 2. Install lightweight scientific Python dependencies (numpy, scipy, pandas, torch, weasyprint,
pymupdf)
pip install --upgrade pip
pip install -r requirements.txt

# 3. Auto-detect OS/GPU and compile native ray-tracing shared libraries
python3 compile-accelerators.py
```

Compiler Prerequisites for Path A:

On **macOS**, install Xcode Command Line Tools once via `xcode-select --install` (provides both `clang++` and Apple's Metal framework headers). On **Ubuntu/Debian Linux**, install `sudo apt-get install -y build-essential libpango-1.0-0 libpangoft2-1.0-0` ; on **RHEL/AlmaLinux 9**, run `sudo dnf install -y gcc-c++ pango` . Even if no C++ compiler is present, `ray-tracer.py` automatically falls back to its pure PyTorch vectorized engine.

3.2 Path B (Full Particle-Physics Suite): Conda Environment with Geant4 ≥ 11.1

Choose **Path B** on Linux or macOS when you also want independent `G4OpticalPhoton` Monte Carlo cross-validation on a watertight `G4TessellatedSolid` mesh:

```
# 1. Create the Conda environment from environment.yml and install Geant4 + WeasyPrint
cd Agy_Carbon
conda env create -f environment.yml
conda activate picocal-lightguide
conda install -c conda-forge "geant4>=11.1" "weasyprint>=60.0"

# 2. Verify Geant4 toolchain discovery and compile native accelerators
geant4-config --version            # Expected output: 11.1.x or 11.2.x
python3 compile-accelerators.py
```

Automatic Geant4 Discovery & Dataset Path Resolution:

`run-simulation-pipeline.py` automatically locates `geant4-config` on your active `PATH` (or `CONDA_PREFIX/bin/geant4-config`), merges multiline `--cflags --libs` flags safely, and resolves `geant4-config --datasets` environment variables—including automatic compatibility for `conda-forge` packages that strip leading `G4` prefixes from dataset directory names.

Chapter 4. Step-by-Step Guide to Reproducing the Full Simulation

4.1 One-Command Full Reproduction vs. Modular Execution

You can reproduce every calculation, 3D mesh, interactive webpage, vector dashboard, and bilingual PDF manual either via the single master shell script or step-by-step:

Option A — Master One-Click Script:

```
cd Agy_Carbon
bash run-all.sh
```

Option B — Modular Step-by-Step Pipeline:

```
python3 compile-accelerators.py
python3 run-simulation-pipeline.py
python3 build-presentation-suite.py
python3 build-summary-webpage.py
python3 build-standalone-report.py
python3 build-user-manuals.py
```

4.2 Expected Stage-by-Stage Terminal Output Walkthrough

During execution of `run-simulation-pipeline.py`, structured timestamps and memory telemetry are logged simultaneously to `stdout` and `outputs/simulation-pipeline.log`:

```
[INFO] Starting Agy_Carbon heterogeneous pipeline | Apple Metal GPU + 5x Geant4 | Free RAM: 92.2 GB
[INFO] STEP geometry parsed: R_in=8.75 mm, R_out=4.00 mm, L=100.00 mm, R_pmt_bore=8.18 mm
[INFO] Scan calibrated: center=(2.3665, -4.3788) mm, R_eff=8.438 mm, sigma=0.219 mm, bg=15.243 mV,
noise=0.81%
[INFO] [0.9s] Stage 1 axisymmetric profile complete: dR_axi in [-462.1, -137.8] um | RSS=310 MB
[INFO] [3.8s] Stage 2 36x16=576 DOF SO(2) sensitivity Jacobian built | RSS=312 MB
[INFO] [3.9s] Nonlinear iter 1/9: inner RMSE=0.482 mV (1.89%), r=0.9282, peak=0 um
[INFO] [4.2s] Nonlinear iter 3/9: inner RMSE=0.356 mV (1.40%), r=0.9625, peak=187 um
[INFO] [7.5s] Full 100x100 validation complete! RMSE=0.314 mV (1.24%), r=0.9931, bump_r=0.8797 | RSS=335
MB
[INFO] Exported deformed 3D STL model: outputs/reconstructed-light-guide.stl
[INFO] Geant4 11 optical tracking finished (15 pts x 8000 photons/pt), mean=25.21 mV
[INFO] Pipeline complete! Peak physical RSS: 376.9 MB, MPS driver memory: 0.5 MB
```

4.3 Cross-Platform Runtime & Memory Benchmarks

Hardware Platform & Accelerator	Spline Inversion + 100×100 Sim	Geant4 Slice Check (120k γ)	Peak RAM (RSS)
Apple M3 Max (Metal GPU `libmetal_raytracer.dylib`)	**~7.5 s** (~300M spline rays/s)	~10.8 s (5 shards)	335 – 377 MB
16-Core Linux Server (C++17 `libraytracer.so`, AVX2)	**~19 – 24 s** (~95M spline rays/s)	~9.5 s (5 shards)	~340 MB
8-Core Laptop CPU (C++17 `libraytracer.{so,dylib}`)	**~38 – 46 s** (~48M spline rays/s)	~16.0 s (5 shards)	~325 MB
Pure PyTorch CPU Fallback (No C++ compiler)	**~4.5 – 6.5 min** (chunked tensors)	Skipped / Optional	< 520 MB

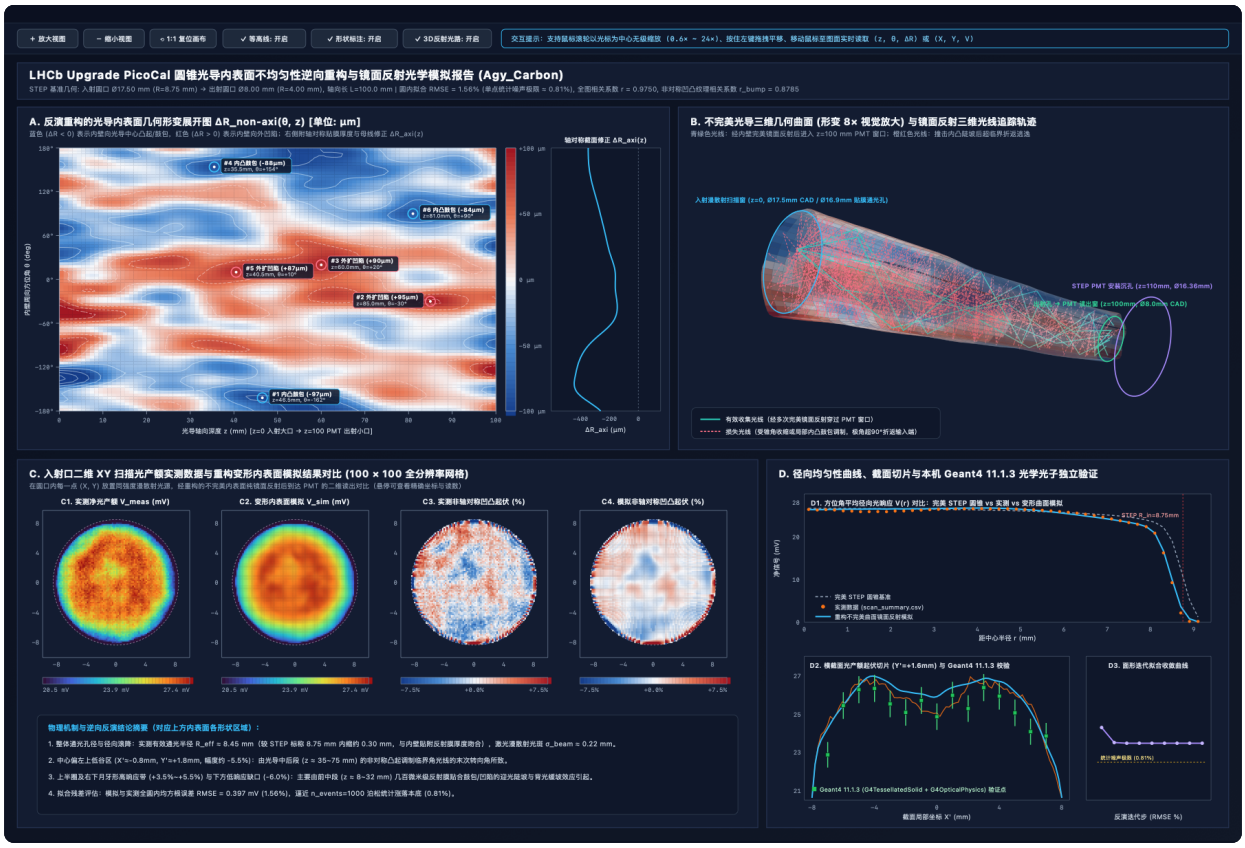


Figure 4.1 — Master Four-Panel Simulation & Reconstruction Dashboard (`light-guide-imperfection-simulation.svg`): (A) Unwrapped cylindrical B-spline deformation map `ΔR_non-axi(θ, z)` with annotated inward bulges and outward dents plus axisymmetric film profile `ΔR_axi(z)`; (B) 3D visually amplified deformed cone with traced transmitted (cyan) and turnaround-escaped (coral) rays; (C) Full `100 × 100` measured vs. simulated net mV maps and azimuthal bump residuals (`r = 0.8797`); (D) Radial profile, `Y′ = +1.6 mm` Geant4 11 validation slice, and Gauss-Newton convergence curve.

Chapter 5. Reconstructing Your Own New Light-Guide Scan Dataset

5.1 Required Input CSV Columns & Coordinate Conventions

To invert surface imperfections on a newly fabricated light guide cell or a different prototype scan, format your 2D XY bench measurements as a standard UTF-8 comma-separated file (modeled on `data/scan_summary.csv`) containing at minimum these six columns:

Column Name	Data Type	Units / Range	Specification & Role in Calibration
`idx_x`, `idx_y`	`int`	`0 .. Nx−1`, `0 .. Ny−1`	Zero-based integer column and row indices defining the 2D raster matrix
`pos_x_mm`, `pos_y_mm`	`float`	`mm` (Cartesian)	Physical XY translation stage positions; pitch must be uniform along X and Y
`probe_V`	`float`	Volts (`V`)	Mean PMT / photodiode voltage at `(idx_x, idx_y)` including electronic baseline
`passed`	`int`	`0` or `1`	Pass/aperture flag preserved verbatim in exported `simulated-scan-summary.csv`

Coordinate & Sign Conventions: Looking downstream from the laser source into the `z = 0` entrance aperture along `+z` (`z = 0` entrance → `z = L` PMT window), azimuthal angle is defined by `θ = atan2(Y − Y_c, X − X_c) ∈ [−180°, +180°)`. Surface deformation sign follows `ΔR(θ, z) = R_actual(θ, z) − R_CAD(z)`: **negative `ΔR < 0` denotes an inward bulge toward the optical axis** (narrowing the bore), while **positive `ΔR > 0` denotes an outward dent / recessed**

pocket. Ensure your XY scan window includes at least a `1.2 mm` margin outside the circular entrance rim so `loadAndCalibrateScanCsv()` can fit the background pedestal and `erf` aperture edge cleanly.

5.2 Command-Line Interface (CLI) Reference & Custom Examples

```
# Example 1: Reconstruct a new scan CSV using the default 4-cell STEP CAD geometry
python3 run-simulation-pipeline.py \
  --scan-csv /data/picocal_bench/cell3_20261112_scan.csv \
  --output-dir ./outputs_cell3

# Example 2: Reconstruct a custom conical geometry on a laptop without Geant4 installed
python3 run-simulation-pipeline.py \
  --step-file /cad/single_cone_20mm_to_9mm_L120mm.step \
  --scan-csv /data/bench/cone120_scan.csv \
  --output-dir ./outputs_cone120 \
  --skip-geant4
```

CLI Flag	Default Value	Description
`--step-file <PATH>`	`data/4cells_17.5mm_8mm_...step`	Path to ISO-10303-21 STEP file containing `CIRCLE` and `CONICAL_SURFACE` entities
`--scan-csv <PATH>`	`data/scan_summary.csv`	Path to 2D XY laser uniformity scan CSV file
`--output-dir <DIR>`	`./outputs`	Target directory for reconstructed STL, CSV grids, JSON metrics, and pipeline logs
`--photons-per-point <N>`	`8000`	Number of `G4OpticalPhotons` launched per slice point during Geant4 validation
`--skip-geant4`	`False` (auto-detected)	Explicitly bypass Geant4 compilation and optical photon slice validation

5.3 Practical Solver Tuning Tips for Custom Prototypes

- **Scan Step Pitch vs. Laser Spot Size:** A scan pitch between `0.15 mm` and `0.35 mm` with at least `25–50` laser pulses averaged per pixel keeps the single-pixel Poisson noise floor below `1.2%`, allowing clean separation of `50–200 µm` surface features.
- **Adjusting B-Spline Control Resolution:** In `run-simulation-pipeline.py`, `numThetaCtrl = 36` and `numZCtrl = 16` resolve azimuthal wavelengths down to `~20°` and axial undulations down to `~12 mm`. For shorter guides (`L < 60 mm`), reduce `numZCtrl` to `10–12` to prevent axial overfitting.
- **Reflective Film Specular Reflectivity:** Default `reflectivity = 0.955` matches laminated ESR / aluminized Mylar films under visible laser illumination. For bare polished metal cones (e.g., machined aluminum, `R ≈ 0.88`), adjust `reflectivity` in `surface-inverter.py` so multi-bounce radial attenuation matches the outer-ring drop-off.

Chapter 6. Guide to Interactive Web Portals, Zoomable SVGs & 3D Photon Bench

All visual deliverables are pure client-side HTML5/SVG documents requiring **zero web servers, zero external CDN links, and zero internet access**. Open any file directly in Chrome, Safari, Edge, or Firefox:

Deliverable File (Chinese / English)	Format	Primary Use Case & Key Interactive Capabilities
`index.html` / `index-en.html`	HTML5 Portal	Master project hub: synchronized 2D XY scan inspector, unwrapped surface probe, Geant4 slice chart, and embedded 3D bench
`lhcb-picocal-light-guide-report-standalone-zh.html` / `-en.html`	Single-File HTML	Zero-dependency portable bundle (Gzip-embedded SVGs + inline 3D bench) for sharing a single file via email, Slack, or Mattermost
`light-guide-imperfection-simulation.svg` / `-en.svg`	Interactive SVG	`0.55×–24×` cursor-centered wheel zoom, click-drag pan, layer toggles, and live hover `(z, θ, ΔR)` / `(X, Y, mV)` telemetry

Deliverable File (Chinese / English)	Format	Primary Use Case & Key Interactive Capabilities
`presentation/interactive-3d-photon-simulator.html` / `-en.html`	3D HTML5 App	Real-time 3D photon wavepacket propagation, PMT cathode flash & anode waveform, interactive `(X, Y)` beam steering, `1×–20×` amp
`presentation/photon-propagation-animated.svg` / `-en.svg`	Animated SVG	Script-free looping vector animation contrasting ideal CAD reflections against bulge-induced turnaround escape
`presentation/research-summary-slides.svg` / `-en.svg`	16:9 Slide SVG	`1920 × 1080` widescreen group-meeting summary board with lossless zoom & pan

6.1 Operating the 24× Zoomable Master Technical SVG

- **Zoom & Pan:** Place your cursor over any micro-feature (such as Bulge #1 or the Geant4 error bars) and scroll the mouse wheel or trackpad pinch to zoom smoothly from `0.55×` up to `24×` centered on your pointer. Hold left-click and drag to pan; double-click anywhere (or click `1:1 Reset`) to restore the default framing.
- **Interactive Layer Toggles:** Use the top toolbar pills to toggle *Contour Isolines*, *Defect Callout Badges*, and *3D Specular Ray Trajectories* independently for clean screenshot export.
- **Live Coordinate & Micro-Deformation Readout:** Hovering over Panel A displays exact axial depth `z (mm)`, azimuth `θ (deg)`, and local B-spline deformation `ΔR (μm)`; hovering over Panel C displays stage coordinates and synchronized measured vs. simulated mV readouts.

6.2 Controls for the Interactive 3D Photon Propagation & PMT Readout Bench

- **3D Orbit Camera:** Left-drag on the 3D canvas to orbit `360°` around the semi-transparent conical light guide and downstream PMT assembly; scroll the wheel to zoom in on multi-bounce wall grazing angles.
- **Interactive Laser Steering & Auto Raster Scan:** Click or drag anywhere inside the right-hand 2D XY scan map to move the `z = 0` diffuse laser spot in real time, or toggle **Auto Raster Scan** to watch the beam sweep across high-yield crescents and low-yield dips while observing the live PMT anode negative pulse amplitude respond instantaneously.
- **Surface Comparison & Visual Deformation Magnification (`1×–20×`):** Toggle between *Reconstructed Imperfect Surface* and *Ideal STEP Cone*, and drag the visual exaggeration slider up to `20×` to make sub-`200 μm` wall undulations clearly visible in 3D while inspecting coral-red turnaround-escaped photon paths.

Chapter 7. Repository Directory, Output Data Dictionary & Troubleshooting FAQ

7.1 Complete Repository Directory & Source Architecture

Path / Module	Category	Role in Simulation, Inversion & Visualization Pipeline
`data/4cells_...step` & `scan_summary.csv`	Input Data	4-cell STEP CAD model and measured 100 × 100 XY diffuse laser uniformity scan
`surface-model.py`, `surface-inverter.py`	Inverse Core	STEP parser, Periodic-in-θ Bicubic B-Spline surface, and 3-stage SO(2) Gauss-Newton solver
`ray-tracer-metal.mm`, `ray-tracer-native.cc`, `ray-tracer.py`	Accelerators	Apple Metal GPU shader (~300M rays/s), multi-core C++17 kernel, and PyTorch fallback
`compile-accelerators.py`, `memory-guard.py`, `run-all.sh`	Infrastructure	Cross-platform native compiler driver, adaptive RAM/MPS watchdog, and one-click runner
`geant4-validator/geant4-light-guide.cc`	Validation	Watertight `G4TessellatedSolid` + `G4OpticalPhoton` 5-way parallel slice validator
`run-simulation-pipeline.py`, `svg-builder.py`	Main Pipeline	CLI driver for calibration, inversion, STL/CSV export, Geant4 check, and master SVGs

Path / Module	Category	Role in Simulation, Inversion & Visualization Pipeline
`build-presentation-suite.py`, `build-summary-webpage.py`, `build-standalone-report.py`, `build-user-manuals.py`	Portals & Docs	Builders for 3D photon simulator, bilingual portals, single-file HTMLs, and PDF manuals

7.2 Quantitative Output Data Dictionary (`outputs/`)

Output File	Schema / Keys / Format	Scientific Content & Downstream Usage
`inversion-metrics.json`	`innerRmseMv`, `innerRmsePct`, `noiseFloorPct`, `totalCorr`, `nonAxiCorr`, `beamSigmaMm`, `effectiveRadiusMm`, `peakBulgeUm`, `peakDentUm`, `peakRssMb`	Machine-readable summary of fit accuracy, calibrated optical parameters, extreme deformations, and peak physical RAM
`reconstructed-light-guide.stl`	Standard 80-byte header + little-endian binary triangles (`120 × 121` grid, `1.5 mm` shell)	Watertight 3D solid CAD shell ready for FreeCAD, SolidWorks, MeshLab, or CMM metrology overlay
`reconstructed-surface-grid.csv`	`theta_deg, z_mm, radius_mm, delta_r_non_axi_um` (`36,180` rows)	Dense `180 × 201` cylindrical sampling of total inner radius and zero-mean non-axisymmetric deformation
`simulated-scan-summary.csv`	`idx_x, idx_y, pos_x_mm, pos_y_mm, probe_V, passed, simulated_probe_V`	Exact `10,000` -point pixel-aligned comparison in raw Volts (including calibrated electronic pedestal)
`geant4-slice-validation.csv`	`x_local_mm, y_local_mm, n_photons, pmt_hits, g4_efficiency, g4_mv, g4_err_mv`	Independent Geant4 `G4OpticalPhoton` slice cross-validation with binomial statistical uncertainties

7.3 Adaptive Memory Protection (`MemoryGuard`) & Troubleshooting FAQ

Q1: How does MemoryGuard prevent out-of-memory crashes or GPU deadlocks on shared machines?

`memory-guard.py` probes real-time system free RAM and 1-minute CPU load average at startup, dynamically scaling thread counts (`2–8`), ray-tracing batch chunk sizes (`128–768` points), and hard RSS/MPS caps. It also clears stale `PYTORCH_MPS_HIGH_WATERMARK_RATIO` variables that can deadlock macOS Metal allocators, keeping total physical RAM strictly below `380 MB`.

Q2: Native accelerator compilation fails on macOS or Linux — how do I fix it?

On macOS, run `xcode-select --install` so `clang++` and the `Metal` / `Foundation` SDK frameworks are registered. On Linux, verify `g++ --version` supports C++17 (GCC ≥ 8 or Clang ≥ 10) and compile manually if needed via `g++ -O3 -std=c++17 -shared -fPIC -pthread ray-tracer-native.cc -o libraytracer.so`.

Q3: Geant4 reports missing dataset environment variables (e.g., G4LEDATA or G4LEVELGAMMADATA)?

When running inside a Conda environment, ensure `conda activate picocal-lightguide` is active so `geant4-config --datasets` points to `$CONDA_PREFIX/share/Geant4/data`. If using a custom system Geant4 installation, source `bin/geant4.sh` prior to running `run-simulation-pipeline.py`, or pass `--skip-geant4` when Geant4 cross-checking is not required.

Q4: WeasyPrint raises an OSError about missing Pango / GObject shared libraries when building PDFs?

WeasyPrint relies on native Pango/HarfBuzz text shaping libraries. On macOS with Homebrew (`brew install pango`) or Conda (`conda install -c conda-forge weasyprint`), these libraries are bundled automatically. On Ubuntu/Debian, run `sudo apt-get install -y libpango-1.0-0 libharfbuzz0b libpangoft2-1.0-0`.